

Practical 1

```
In [3]: #integer
age=17
print("Type:", type(age))

#float
a=678.9
print("Type:", type(a))

# string
name="yagasri"
print("Type:", type(name))

# Boolean
is_name=True
print("Type:", type(is_name))

# list
a=[1,2,3,4]
print("Type:", type(a))

# Tuple
a=(1,2,3,4)
print("Type:", type(a))

# dictionary
marks={"Tamil":"100", "English":"100"}
print("Type:", type(marks))

# Set
a={2,3,4}
print("Type:", type(a))
```

```
Type: <class 'int'>
Type: <class 'float'>
Type: <class 'str'>
Type: <class 'type'>
Type: <class 'list'>
Type: <class 'tuple'>
Type: <class 'dict'>
Type: <class 'set'>
```

Practical 2

```
In [2]: print("--- 1. Check Positive Number ---")
num1 = int(input("Enter a number: "))
if num1 > 0:
```

```
    print(num1, "is Positive")
else:
    print(num1, "is Negative or Zero")

print("\n--- 2. Check Even or Odd ---")
num2 = int(input("Enter another number: "))
if num2 % 2 == 0:
    print(num2, "is Even")
else:
    print(num2, "is Odd")

print("\n--- 3. Assign Grade ---")
marks = int(input("Enter student marks out of 100: "))
if marks >= 90:
    grade = "A"
elif marks >= 75:
    grade = "B"
elif marks >= 50:
    grade = "C"
else:
    grade = "Fail"
print("Grade:", grade)

print("\n--- 4. Login Verification---")
username = input("Enter username: ")
password = input("Enter password: ")
if username == "admin":
    if password == "1234":
        print("Login Successful")
    else:
        print("Wrong Password")
else:
    print("Wrong Username")

print("\n--- 5. Voting Eligibility ---")
age = int(input("Enter your age: "))
print("Eligible to vote" if age >= 18 else print("Not Eligible to vote"))

print("\n--- 6. For Loop: 1 to 5 ---")
for i in range(1, 6):
    print(i, end=" ")

print("\n\n--- 7. While Loop: 1 to 5 ---")
j = 1
while j <= 5:
    print(j, end=" ")
    j += 1

print("\n\n--- 8. Break Statement ---")
for k in range(1, 10):
    if k == 5:
        break
    print(k, end=" ")

print("\n\n--- 9. Continue Statement ---")
for k in range(1, 10):
```

```
    if k == 5:
        continue
    print(k, end=" ")

print("\n\n--- 10. Pass Statement ---")
for k in range(1, 6):
    if k == 3:
        pass
    else:
        print(k, end=" ")

print("\n\n--- 11. Match-Case for Day ---")
day = int(input("Enter day number 1-7: "))
match day:
    case 1: print("Monday")
    case 2: print("Tuesday")
    case 3: print("Wednesday")
    case 4: print("Thursday")
    case 5: print("Friday")
    case 6: print("Saturday")
    case 7: print("Sunday")
    case _: print("Invalid day number")

print("\n\n--- Program End ---")
```

```

--- 1. Check Positive Number ---
Enter a number: 3
3 is Positive

--- 2. Check Even or Odd ---
Enter another number: 3
3 is Odd

--- 3. Assign Grade ---
Enter student marks out of 100: 100
Grade: A

--- 4. Login Verification---
Enter username: 1234
Enter password: yagasri
Wrong Username

--- 5. Voting Eligibility ---
Enter your age: 17
Not Eligible to vote

--- 6. For Loop: 1 to 5 ---
1 2 3 4 5

--- 7. While Loop: 1 to 5 ---
1 2 3 4 5

--- 8. Break Statement ---
1 2 3 4

--- 9. Continue Statement ---
1 2 3 4 6 7 8 9

--- 10. Pass Statement ---
1 2 4 5

--- 11. Match-Case for Day ---
Enter day number 1-7: 4
Thursday

--- Program End ---

```

Practical 3

```

In [3]: def greet():
        print("Hello! Welcome to Python Functions.")
        greet()

# 2. Function with arguments
def add(a, b):
    print("Sum =", a + b)
add(10, 20)

# 3. Function without arguments but with return value

```

```
def get_welcome():  
    return "Welcome to learn python."  
a = get_welcome()  
print(a)  
  
# 4. Function with arguments and return value  
def multiply(a, b):  
    return a * b  
print("Product =", multiply(5, 4))  
  
# 5. Function with default argument  
def country(name="India"):  
    print("Country:", name)  
country()  
country("Japan")  
  
# 6. Function with keyword arguments  
def student(name, age):  
    print("Name:", name)  
    print("Age:", age)  
student(age=18, name="Ravi")  
  
# 7. Function with variable-length arguments  
def total_marks(*marks):  
    total = sum(marks)  
    print("Total Marks =", total)  
total_marks(80, 90, 85, 95)  
  
# 8. Lambda function  
square = lambda x: x * x  
print("Square =", square(6))  
  
# 9. Recursive function for factorial  
def factorial(n):  
    if n == 0 or n == 1:  
        return 1  
    else:  
        return n * factorial(n - 1)  
print("Factorial =", factorial(5))  
  
# 10. Nested function  
def outer():  
    print("Outer Function")  
  
    def inner():  
        print("Inner Function")  
  
    inner()  
outer()
```

Hello! Welcome to Python Functions.
Sum = 30
Welcome to learn python.
Product = 20
Country: India
Country: Japan
Name: Ravi
Age: 18
Total Marks = 350
Square = 36
Factorial = 120
Outer Function
Inner Function

Practical 4

```
In [4]: # Create a list
nums = [1, 2, 3, 4, 5]

print("Original List:", nums)
print("First Element:", nums[0])
print("Last Element:", nums[-1])

# Append
nums.append(6)
print("After Append:", nums)

# Insert
nums.insert(2, 10)
print("After Insert:", nums)

# Remove
nums.remove(10)
print("After Remove:", nums)

# Length
print("Length:", len(nums))

# Traverse
print("List Elements:")
for i in nums:
    print(i)

# List Comprehension
square = [x*x for x in nums]
even = [x for x in nums if x % 2 == 0]
odd = [x for x in nums if x % 2 != 0]
double = [x*2 for x in nums]

matrix = [[1, 2], [3, 4]]
flat = [x for row in matrix for x in row]

print("Squares:", square)
print("Even:", even)
```

```
print("Odd:", odd)
print("Double:", double)
print("Flattened List:", flat)
```

Original List: [1, 2, 3, 4, 5]
 First Element: 1
 Last Element: 5
 After Append: [1, 2, 3, 4, 5, 6]
 After Insert: [1, 2, 10, 3, 4, 5, 6]
 After Remove: [1, 2, 3, 4, 5, 6]
 Length: 6
 List Elements:
 1
 2
 3
 4
 5
 6
 Squares: [1, 4, 9, 16, 25, 36]
 Even: [2, 4, 6]
 Odd: [1, 3, 5]
 Double: [2, 4, 6, 8, 10, 12]
 Flattened List: [1, 2, 3, 4]

Practical 5

```
In [5]: # Tuple
t = (10, 20, 30, 40, 20)

print("Tuple:", t)
print("First Element:", t[0])
print("Last Element:", t[-1])
print("Length:", len(t))
print("Slicing:", t[1:4])
print("Count of 20:", t.count(20))
print("Index of 30:", t.index(30))

print("Tuple Elements:")
for i in t:
    print(i)

t2 = (50, 60)
print("Concatenated Tuple:", t + t2)

# Set
s = {1, 2, 3, 4}

print("Original Set:", s)

s.add(5)
print("After Add:", s)

s.remove(2)
print("After Remove:", s)
```

```

print("Length:", len(s))

print("Set Elements:")
for i in s:
    print(i)

s2 = {3, 4, 5, 6}

print("Union:", s | s2)
print("Intersection:", s & s2)
print("Difference:", s - s2)
print("Symmetric Difference:", s ^ s2)

```

```

Tuple: (10, 20, 30, 40, 20)
First Element: 10
Last Element: 20
Length: 5
Slicing: (20, 30, 40)
Count of 20: 2
Index of 30: 2
Tuple Elements:
10
20
30
40
20
Concatenated Tuple: (10, 20, 30, 40, 20, 50, 60)
Original Set: {1, 2, 3, 4}
After Add: {1, 2, 3, 4, 5}
After Remove: {1, 3, 4, 5}
Length: 4
Set Elements:
1
3
4
5
Union: {1, 3, 4, 5, 6}
Intersection: {3, 4, 5}
Difference: {1}
Symmetric Difference: {1, 6}

```

Practical 6

```

In [6]: # Dictionary Operations
student = {
    "Name": "Yagasri",
    "Age": 17,
    "Course": "BSc DS & AI"
}

print("Original Dictionary:", student)

# Access value

```

```
print("Name:", student["Name"])

# Add new key
student["College"] = "SDNB Vaishnav College"

# Update value
student["Age"] = 18

# Remove key
student.pop("Course")

# Length
print("Length:", len(student))

# Keys
print("Keys:", student.keys())

# Values
print("Values:", student.values())

# Items
print("Items:", student.items())

# Traverse dictionary
print("Dictionary:")
for key, value in student.items():
    print(key, ":", value)

# Check key
if "Age" in student:
    print("Age key exists")

# Copy dictionary
copy_student = student.copy()
print("Copied Dictionary:", copy_student)

# Clear another dictionary
d = {"A": 1, "B": 2}
d.clear()
print("Cleared Dictionary:", d)

print("Updated Dictionary:", student)
```

```

Original Dictionary: {'Name': 'Yagasri', 'Age': 17, 'Course': 'BSc DS & AI'}
Name: Yagasri
Length: 3
Keys: dict_keys(['Name', 'Age', 'College'])
Values: dict_values(['Yagasri', 18, 'SDNB Vaishnav College'])
Items: dict_items([('Name', 'Yagasri'), ('Age', 18), ('College', 'SDNB Vaishnav College')])
Dictionary:
Name : Yagasri
Age : 18
College : SDNB Vaishnav College
Age key exists
Copied Dictionary: {'Name': 'Yagasri', 'Age': 18, 'College': 'SDNB Vaishnav College'}
Cleared Dictionary: {}
Updated Dictionary: {'Name': 'Yagasri', 'Age': 18, 'College': 'SDNB Vaishnav College'}

```

Practical 7

```

In [7]: # File Handling

# Write to file
file = open("sample.txt", "w")
file.write("Welcome to Python\n")
file.write("File Handling Example\n")
file.close()

# Read file
file = open("sample.txt", "r")
print("File Contents:")
print(file.read())
file.close()

# Append data
file = open("sample.txt", "a")
file.write("This is appended text.\n")
file.close()

# Read updated file
file = open("sample.txt", "r")
print("Updated File Contents:")
print(file.read())
file.close()

```

File Contents:
 Welcome to Python
 File Handling Example

Updated File Contents:
 Welcome to Python
 File Handling Example
 This is appended text.

Practical 8

```
In [8]: import numpy as np

# Create two NumPy arrays
a = np.array([10, 20, 30, 40, 50])
b = np.array([2, 4, 5, 8, 10])

print("Array A:", a)
print("Array B:", b)

# Array operations
print("Addition:", a + b)
print("Subtraction:", a - b)
print("Multiplication:", a * b)
print("Division:", a / b)

# Indexing
print("First element of A:", a[0])

# Slicing
print("Sliced array A:", a[1:4])

# Reshape
c = np.array([1, 2, 3, 4, 5, 6])
d = c.reshape(2, 3)

print("Reshaped array:")
print(d)
print("Shape:", d.shape)
print("Dimension:", d.ndim)
print("Size:", d.size)

# Aggregation operations
print("Sum:", np.sum(a))
print("Mean:", np.mean(a))
print("Maximum:", np.max(a))
print("Minimum:", np.min(a))

# Universal functions
print("Square root:", np.sqrt(a))
print("Exponential:", np.exp(a))

# Sorting
print("Sorted A:", np.sort(a))

# Concatenation
result = np.concatenate((a, b))
print("Concatenated array:", result)
```

```

Array A: [10 20 30 40 50]
Array B: [ 2  4  5  8 10]
Addition: [12 24 35 48 60]
Subtraction: [ 8 16 25 32 40]
Multiplication: [ 20  80 150 320 500]
Division: [5.  5.  6.  5.  5.]
First element of A: 10
Sliced array A: [20 30 40]
Reshaped array:
[[1 2 3]
 [4 5 6]]
Shape: (2, 3)
Dimension: 2
Size: 6
Sum: 150
Mean: 30.0
Maximum: 50
Minimum: 10
Square root: [3.16227766 4.47213595 5.47722558 6.32455532 7.07106781]
Exponential: [2.20264658e+04 4.85165195e+08 1.06864746e+13 2.35385267e+17
 5.18470553e+21]
Sorted A: [10 20 30 40 50]
Concatenated array: [10 20 30 40 50  2  4  5  8 10]

```

Practical 9

```

In [9]: import numpy as np
        #Create a 1D NumPy array
        arr1 = np.array([10, 20, 30, 40, 50, 60])
        print("1D Array:", arr1)

        #Access specific elements using indexing
        print("\nElement at index 0:", arr1[0])
        print("Element at index 3:", arr1[3])

        #Extract subsets using slicing
        print("\nSliced array [1:4]:", arr1[1:4])
        print("Sliced array [2:]:", arr1[2:])

        #Create a 2D NumPy array
        arr2d = np.array([[1, 2, 3], [4, 5, 6]])
        print("\n2D Array:")
        print(arr2d)

        #Access individual elements from 2D array
        print("\nElement at row 0, col 1:", arr2d[0, 1])
        print("Element at row 1, col 2:", arr2d[1, 2])

        #Extract rows and columns from 2D array
        print("\nFirst row:", arr2d[0])
        print("Second column:", arr2d[:, 1])

        #Create another 1D array for reshaping
        arr3 = np.array([1, 2, 3, 4, 5, 6])

```

```

#Reshape into 2x3 matrix
reshaped_2x3 = arr3.reshape(2, 3)
print("\nReshaped to 2x3:")
print(reshaped_2x3)

#Reshape into 3x2 matrix
reshaped_3x2 = arr3.reshape(3, 2)
print("\nReshaped to 3x2:")
print(reshaped_3x2)

#Convert back to 1D using flatten()
flattened = reshaped_2x3.flatten()
print("\nFlattened array:", flattened)

#Display array attributes
print("\nShape:", arr2d.shape)
print("Dimensions:", arr2d.ndim)
print("Size:", arr2d.size)

```

1D Array: [10 20 30 40 50 60]

Element at index 0: 10

Element at index 3: 40

Sliced array [1:4]: [20 30 40]

Sliced array [2:]: [30 40 50 60]

2D Array:

```
[[1 2 3]
 [4 5 6]]
```

Element at row 0, col 1: 2

Element at row 1, col 2: 6

First row: [1 2 3]

Second column: [2 5]

Reshaped to 2x3:

```
[[1 2 3]
 [4 5 6]]
```

Reshaped to 3x2:

```
[[1 2]
 [3 4]
 [5 6]]
```

Flattened array: [1 2 3 4 5 6]

Shape: (2, 3)

Dimensions: 2

Size: 6

Practical 10

```
In [10]: import numpy as np

#Create a NumPy array with numeric elements
data = np.array([12, 15, 18, 21, 25, 30, 35, 40, 45, 50])

#Display the array elements
print("Array:", data)

#Sum
print("Sum:", np.sum(data))

#Mean
print("Mean:", np.mean(data))

#Median
print("Median:", np.median(data))

#Standard Deviation and Variance
print("Standard Deviation:", np.std(data))
print("Variance:", np.var(data))

#Min and Max values
print("Minimum:", np.min(data))
print("Maximum:", np.max(data))

#Indices of Min and Max
print("Index of Minimum:", np.argmin(data))
print("Index of Maximum:", np.argmax(data))

#Range
print("Range:", np.max(data) - np.min(data))

#Percentiles: 25th, 50th, 75th
print("25th Percentile:", np.percentile(data, 25))
print("50th Percentile:", np.percentile(data, 50))
print("75th Percentile:", np.percentile(data, 75))

#Cumulative Sum
print("Cumulative Sum:", np.cumsum(data))

# Sorted array
print("Sorted Array:", np.sort(data))
```

Array: [12 15 18 21 25 30 35 40 45 50]
 Sum: 291
 Mean: 29.1
 Median: 27.5
 Standard Deviation: 12.41329931968129
 Variance: 154.08999999999997
 Minimum: 12
 Maximum: 50
 Index of Minimum: 0
 Index of Maximum: 9
 Range: 38
 25th Percentile: 18.75
 50th Percentile: 27.5
 75th Percentile: 38.75
 Cumulative Sum: [12 27 45 66 91 121 156 196 241 291]
 Sorted Array: [12 15 18 21 25 30 35 40 45 50]

Practical 11

```

In [11]: # Pandas Series and DataFrame Operations
import pandas as pd

# -----SERIES-----

# Creating a series
series1 = pd.Series([10,20,30,40,50])

print("Pandas Series:")
print(series1)

# Accesssing Series Elements
print("\nFirst Element:")
print(series1[0])

# Series with Custom Index
series2 = pd.Series(
    [85,90,95],
    index=["Math", "Science", "English"]
)

print("\nSeries with Custom Index:")

#-----DATAFRAME-----

# Creating DataFrame using Dictionary
student = {
    "ID": [101,102,103],
    "Name": ["Sakshi", "Rahul", "Ankit"],
    "Marks": [95,88,99]
}
  
```

```
df = pd.DataFrame(student)

print('\nDataFrame:')
print(df)

# Display First rows
print("\nFirst Two Rows:")
print(df.head(2))

# Display Last rows
print("\nLast Two Rows:")
print(df.tail(2))

# Display Column Names
print("\nColumn Names:")
print(df.columns)

# Accessing Single Column
print("\nMarks Column:")
print(df["Marks"])

# Accessing Multiple Columns
print("\nName and Marks Columns:")
print(df[["Name", "Marks"]])

# Accessing Rows using loc
print("\nFirst Row using loc:")
print(df.loc[1])

# Accessing Rows using iloc
print("\nSecond Row using iloc:")
print(df.iloc[0])

# Adding New Column
df["Grade"] = ["A", "B", "A"]

print("\nAfter Adding Grade Column:")
print(df)

#Updating Values
df.loc[1, "Marks"] = 92

print("\nAfter Updating Marks:")
print(df)

#Deleting Column
df.drop("Grade", axis=1, inplace=True)

print("\nAfter Removing Grade Column:")
print(df)

# DataFrame Information
print("\nDataFrame Information:")
print(df.info())
```

```
# Statistical summary  
print("\nStatistical Sumarry:")  
print(df.describe)
```

Pandas Series:

```
0    10
1    20
2    30
3    40
4    50
```

dtype: int64

First Element:

10

Series with Custom Index:

DataFrame:

	ID	Name	Marks
0	101	Sakshi	95
1	102	Rahul	88
2	103	Ankit	99

First Two Rows:

	ID	Name	Marks
0	101	Sakshi	95
1	102	Rahul	88

Last Two Rows:

	ID	Name	Marks
1	102	Rahul	88
2	103	Ankit	99

Column Names:

Index(['ID', 'Name', 'Marks'], dtype='str')

Marks Column:

```
0    95
1    88
2    99
```

Name: Marks, dtype: int64

Name and Marks Columns:

	Name	Marks
0	Sakshi	95
1	Rahul	88
2	Ankit	99

First Row using loc:

```
ID      102
Name    Rahul
Marks    88
Name: 1, dtype: object
```

Second Row using iloc:

```
ID      101
Name    Sakshi
Marks    95
Name: 0, dtype: object
```

After Adding Grade Column:

	ID	Name	Marks	Grade
0	101	Sakshi	95	A
1	102	Rahul	88	B
2	103	Ankit	99	A

After Updating Marks:

	ID	Name	Marks	Grade
0	101	Sakshi	95	A
1	102	Rahul	92	B
2	103	Ankit	99	A

After Removing Grade Column:

	ID	Name	Marks
0	101	Sakshi	95
1	102	Rahul	92
2	103	Ankit	99

DataFrame Information:

```
<class 'pandas.DataFrame'>
RangeIndex: 3 entries, 0 to 2
Data columns (total 3 columns):
#   Column  Non-Null Count  Dtype
---  -
0    ID      3 non-null        int64
1   Name     3 non-null        str
2   Marks    3 non-null        int64
dtypes: int64(2), str(1)
memory usage: 204.0 bytes
None
```

Statistical Sumarry:

```
<bound method NDFrame.describe of      ID      Name  Marks
0  101  Sakshi     95
1  102   Rahul     92
2  103   Ankit     99>
```

Practical 12

In [3]: *#Handling Missing Data using Pandas*

```
import pandas as pd
import numpy as np

# Create a DataFrame containing missing values
data = {
    'Name': ['Arun', 'Bala', 'Chitra', 'Divya', 'Elango'],
    'Marks': [85, np.nan, 78, np.nan, 92],
    'Age': [20, 21, np.nan, 22, 21],
    'City': ['Chennai', 'Madurai', np.nan, 'Salem', 'Trichy']
}
df = pd.DataFrame(data)

# Display the original DataFrame
```

```
print("Original DataFrame:")
print(df)

# Check missing values using isnull()
print("\nMissing values (True/False):")
print(df.isnull())

# Count missing values in each column
print("\nCount of missing values per column:")
print(df.isnull().sum())

# Fill all missing values with a default value using fillna()
df_filled_default = df.fillna("Unknown")
print("\nAfter fillna() with default value 'Unknown':")
print(df_filled_default)

# Fill missing values in Marks column with column mean using fillna()
df['Marks'] = df['Marks'].fillna(df['Marks'].mean())
print("\nAfter fillna() - Marks column filled with mean:")
print(df)

# Forward fill - CORRECTED
df_ffill = df.ffill()
print("\nAfter ffill() - Forward Fill:")
print(df_ffill)

# Backward fill - CORRECTED
df_bfill = df.bfill()
print("\nAfter bfill() - Backward Fill:")
print(df_bfill)

# Remove rows with missing values using dropna()
df_dropped = df.dropna()
print("\nAfter dropna() - rows with missing values removed:")
print(df_dropped)

# Replace a specified value using replace()
df_replaced = df.replace("Chennai", "Coimbatore")
print("\nAfter replace() - 'Chennai' replaced with 'Coimbatore':")
print(df_replaced)

# Display DataFrame info
print("\nDataFrame Info:")
df.info()
```

Original DataFrame:

	Name	Marks	Age	City
0	Arun	85.0	20.0	Chennai
1	Bala	NaN	21.0	Madurai
2	Chitra	78.0	NaN	NaN
3	Divya	NaN	22.0	Salem
4	Elango	92.0	21.0	Trichy

Missing values (True/False):

	Name	Marks	Age	City
0	False	False	False	False
1	False	True	False	False
2	False	False	True	True
3	False	True	False	False
4	False	False	False	False

Count of missing values per column:

```
Name      0
Marks     2
Age       1
City      1
dtype: int64
```

After fillna() with default value 'Unknown':

	Name	Marks	Age	City
0	Arun	85.0	20.0	Chennai
1	Bala	Unknown	21.0	Madurai
2	Chitra	78.0	Unknown	Unknown
3	Divya	Unknown	22.0	Salem
4	Elango	92.0	21.0	Trichy

After fillna() - Marks column filled with mean:

	Name	Marks	Age	City
0	Arun	85.0	20.0	Chennai
1	Bala	85.0	21.0	Madurai
2	Chitra	78.0	NaN	NaN
3	Divya	85.0	22.0	Salem
4	Elango	92.0	21.0	Trichy

After ffill() - Forward Fill:

	Name	Marks	Age	City
0	Arun	85.0	20.0	Chennai
1	Bala	85.0	21.0	Madurai
2	Chitra	78.0	21.0	Madurai
3	Divya	85.0	22.0	Salem
4	Elango	92.0	21.0	Trichy

After bfill() - Backward Fill:

	Name	Marks	Age	City
0	Arun	85.0	20.0	Chennai
1	Bala	85.0	21.0	Madurai
2	Chitra	78.0	22.0	Salem
3	Divya	85.0	22.0	Salem
4	Elango	92.0	21.0	Trichy

After dropna() - rows with missing values removed:

	Name	Marks	Age	City
0	Arun	85.0	20.0	Chennai
1	Bala	85.0	21.0	Madurai
3	Divya	85.0	22.0	Salem
4	Elango	92.0	21.0	Trichy

After replace() - 'Chennai' replaced with 'Coimbatore':

	Name	Marks	Age	City
0	Arun	85.0	20.0	Coimbatore
1	Bala	85.0	21.0	Madurai
2	Chitra	78.0	NaN	NaN
3	Divya	85.0	22.0	Salem
4	Elango	92.0	21.0	Trichy

DataFrame Info:

```
<class 'pandas.DataFrame'>
RangeIndex: 5 entries, 0 to 4
Data columns (total 4 columns):
#   Column   Non-Null Count  Dtype
---  -
0   Name     5 non-null      str
1   Marks    5 non-null      float64
2   Age      4 non-null      float64
3   City     4 non-null      str
dtypes: float64(2), str(2)
memory usage: 292.0 bytes
```

```
In [2]: # Grouping and Aggregation Data using pandas

import pandas as pd

# Creating DataFrame
data = {
    "Department": ["HR", "IT", "HR", "IT", "Sales", "Sales"],
    "Employee": ["Amit", "Rahul", "Priya", "Ankit", "Sakshi", "Miya"],
    "Salary": [30000, 50000, 35000, 60000, 45000, 40000],
    "Experience": [2, 5, 3, 6, 4, 2]
}

df = pd.DataFrame(data)

print("Original DataFrame:")
print(df)

# Grouping by Department
grouped = df.groupby("Department")

# Sum Aggregation
print("\nSum of Salary by Department:")
print(grouped["Salary"].sum())

# Mean Aggregation
print("\nAverage Salary by Department:")
print(grouped["Salary"].mean())

# Maximum Salary
```

```
print("\nMaximum Salary by Department:")
print(grouped["Salary"].max())

# Minimum Salary
print("\nMinimum Salary by Department:")
print(grouped["Salary"].min())

# Multiple Aggregations
print("\nMultiple Aggregations:")
print(
    grouped["Salary"].agg(
        ["sum", "mean", "max", "min"]
    )
)

# Grouping Multiple Columns
print("\nGrouping by Department and Experience:")

group_multi = df.groupby(
    ["Department", "Experience"]
)["Salary"].sum()

print(group_multi)

# Iterating Groups
print("\nDisplaying Each Group:")

for name, group in grouped:
    print("\nDepartment:", name)
    print(group)

# Sorting Values
print("\nSorted DataFrame by Salary:")

sorted_df = df.sort_values(by="Salary")
print(sorted_df)
```

Original DataFrame:

	Department	Employee	Salary	Experience
0	HR	Amit	30000	2
1	IT	Rahul	50000	5
2	HR	Priya	35000	3
3	IT	Ankit	60000	6
4	Sales	Sakshi	45000	4
5	Sales	Miya	40000	2

Sum of Salary by Department:

Department

HR 65000

IT 110000

Sales 85000

Name: Salary, dtype: int64

Average Salary by Department:

Department

HR 32500.0

IT 55000.0

Sales 42500.0

Name: Salary, dtype: float64

Maximum Salary by Department:

Department

HR 35000

IT 60000

Sales 45000

Name: Salary, dtype: int64

Minimum Salary by Department:

Department

HR 30000

IT 50000

Sales 40000

Name: Salary, dtype: int64

Multiple Aggregations:

	sum	mean	max	min
Department				
HR	65000	32500.0	35000	30000
IT	110000	55000.0	60000	50000
Sales	85000	42500.0	45000	40000

Grouping by Department and Experience:

Department Experience

HR 2 30000

3 35000

IT 5 50000

6 60000

Sales 2 40000

4 45000

Name: Salary, dtype: int64

Displaying Each Group:

Department: HR

	Department	Employee	Salary	Experience
0	HR	Amit	30000	2
2	HR	Priya	35000	3

Department: IT

	Department	Employee	Salary	Experience
1	IT	Rahul	50000	5
3	IT	Ankit	60000	6

Department: Sales

	Department	Employee	Salary	Experience
4	Sales	Sakshi	45000	4
5	Sales	Miya	40000	2

Sorted DataFrame by Salary:

	Department	Employee	Salary	Experience
0	HR	Amit	30000	2
2	HR	Priya	35000	3
5	Sales	Miya	40000	2
4	Sales	Sakshi	45000	4
1	IT	Rahul	50000	5
3	IT	Ankit	60000	6

PRACTICAL 14

In [16]: *# Merge and Join Dataframes using Pandas*

```
import pandas as pd

# ----- CONCATENATE OPERATIONS -----
# (All columns name should be same)

# Creating First DataFrame
df1 = pd.DataFrame({
    "ID": [101, 102, 103, 105],
    "Marks": [95, 88, 90, 85]
}, index=[0, 1, 2, 3])

print("\nDataFrame 1:")
print(df1)

# Creating Second DataFrame
df2 = pd.DataFrame({
    "ID": [201, 202, 203, 205],
    "Marks": [85, 78, 80, 65]
}, index=[4, 5, 6, 7])

print("\nDataFrame 2:")
print(df2)

# Concatenate DataFrames
frames = [df1, df2]
result = pd.concat(frames)
```

```

print("Concatenate DataFrames:")
print(result)

# ----- MERGE OPERATIONS -----

# Creating First DataFrame
df1 = pd.DataFrame({
    "ID": [101, 102, 103, 104],
    "Name": ["Sakshi", "Rahul", "Ankit", "Priya"]
})

print("DataFrame 1:")
print(df1)

# Creating Second DataFrame
df2 = pd.DataFrame({
    "ID": [101, 102, 103, 105],
    "Marks": [95, 88, 90, 85]
}, index=[0, 1, 2, 3])

print("\nDataFrame 2:")
print(df2)

# Merge and Join DataFrames using Pandas

import pandas as pd

# ----- CONCATENATE OPERATIONS ----- (all columns name should

# Creating First DataFrame
df1 = pd.DataFrame({
    "ID": [101, 102, 103, 105],
    "Marks": [95, 88, 90, 85]
}, index=[0, 1, 2, 3])

print("\nDataFrame 1:")
print(df1)

# Creating Second DataFrame
df2 = pd.DataFrame({
    "ID": [201, 202, 203, 205],
    "Marks": [85, 78, 80, 65]
}, index=[4, 5, 6, 7])

print("\nDataFrame 2:")
print(df2)

frames = [df1, df2]
result = pd.concat(frames)
print("concatenate dataframes")
print(result)

# ----- MERGE OPERATIONS ----- ( at least one column s

# Creating First DataFrame

```

```
df1 = pd.DataFrame({
    "ID": [101, 102, 103, 104],
    "Name": ["Sakshi", "Rahul", "Ankit", "Priya"]
})

print("DataFrame 1:")
print(df1)

# Creating Second DataFrame
df2 = pd.DataFrame({
    "ID": [101, 102, 103, 105],
    "Marks": [95, 88, 90, 85]
}, index=[0, 1, 2, 3])

print("\nDataFrame 2:")
print(df2)

# Inner Merge
inner_merge = pd.merge(df1, df2, on="ID", how="inner")

print("\nInner Merge:")
print(inner_merge)

# Left Merge
left_merge = pd.merge(df1, df2, on="ID", how="left")

print("\nLeft Merge:")
print(left_merge)

# Right Merge
right_merge = pd.merge(df1, df2, on="ID", how="right")

print("\nRight Merge:")
print(right_merge)

# Outer Merge
outer_merge = pd.merge(df1, df2, on="ID", how="outer")

print("\nOuter Merge:")
print(outer_merge)

# ----- JOIN OPERATIONS ----- ( columns names in both

# Creating DataFrames for Join
df3 = pd.DataFrame({
    "Name": ["Amit", "Riya", "Karan"]
}, index=[1, 2, 3])

df4 = pd.DataFrame({
    "Salary": [30000, 40000, 50000]
}, index=[1, 2, 3])

print("\nDataFrame 3:")
print(df3)

print("\nDataFrame 4:")
```

```
print(df4)

# Joining DataFrames
joined_df = df3.join(df4)

print("\nJoined DataFrame:")
print(joined_df)

# Joining with Different Join Types
left_join = df3.join(df4, how="left")

print("\nLeft Join:")
print(left_join)

right_join = df3.join(df4, how="right")

print("\nRight Join:")
print(right_join)

outer_join = df3.join(df4, how="outer")

print("\nOuter Join:")
print(outer_join)
```

DataFrame 1:

	ID	Marks
0	101	95
1	102	88
2	103	90
3	105	85

DataFrame 2:

	ID	Marks
4	201	85
5	202	78
6	203	80
7	205	65

Concatenate DataFrames:

	ID	Marks
0	101	95
1	102	88
2	103	90
3	105	85
4	201	85
5	202	78
6	203	80
7	205	65

DataFrame 1:

	ID	Name
0	101	Sakshi
1	102	Rahul
2	103	Ankit
3	104	Priya

DataFrame 2:

	ID	Marks
0	101	95
1	102	88
2	103	90
3	105	85

DataFrame 1:

	ID	Marks
0	101	95
1	102	88
2	103	90
3	105	85

DataFrame 2:

	ID	Marks
4	201	85
5	202	78
6	203	80
7	205	65

concatenate dataframes

	ID	Marks
0	101	95
1	102	88
2	103	90
3	105	85

```

4 201      85
5 202      78
6 203      80
7 205      65

```

DataFrame 1:

```

      ID  Name
0  101  Sakshi
1  102   Rahul
2  103   Ankit
3  104   Priya

```

DataFrame 2:

```

      ID  Marks
0  101     95
1  102     88
2  103     90
3  105     85

```

Inner Merge:

```

      ID  Name  Marks
0  101  Sakshi    95
1  102   Rahul    88
2  103   Ankit    90

```

Left Merge:

```

      ID  Name  Marks
0  101  Sakshi  95.0
1  102   Rahul  88.0
2  103   Ankit  90.0
3  104   Priya   NaN

```

Right Merge:

```

      ID  Name  Marks
0  101  Sakshi    95
1  102   Rahul    88
2  103   Ankit    90
3  105    NaN    85

```

Outer Merge:

```

      ID  Name  Marks
0  101  Sakshi  95.0
1  102   Rahul  88.0
2  103   Ankit  90.0
3  104   Priya   NaN
4  105    NaN  85.0

```

DataFrame 3:

```

      Name
1    Amit
2    Riya
3    Karan

```

DataFrame 4:

```

      Salary
1    30000
2    40000

```

3 50000

Joined DataFrame:

	Name	Salary
1	Amit	30000
2	Riya	40000
3	Karan	50000

Left Join:

	Name	Salary
1	Amit	30000
2	Riya	40000
3	Karan	50000

Right Join:

	Name	Salary
1	Amit	30000
2	Riya	40000
3	Karan	50000

Outer Join:

	Name	Salary
1	Amit	30000
2	Riya	40000
3	Karan	50000

Practical 15

```
In [14]: # simple Plots using Matplotlib

import matplotlib.pyplot as plt

#-----LINE PLOT-----

x = [1,2,3,4,5]
y = [10,20,30,40,50]

plt.figure(figsize=(5,4))

plt.plot(x,y)

plt.title("Line Plot")
plt.xlabel("X Values")
plt.ylabel("Y Values")

plt.show()

#-----BAR GRAPH-----

students = ["A", "B", "C", "D"]
marks = [85,90,75,88]

plt.figure(figsize=(5,4))
```

```
plt.bar(students,marks)

plt.title("Bar Graph")
plt.xlabel("Students")
plt.ylabel("Marks")

plt.show()

#-----HISTOGRAM-----

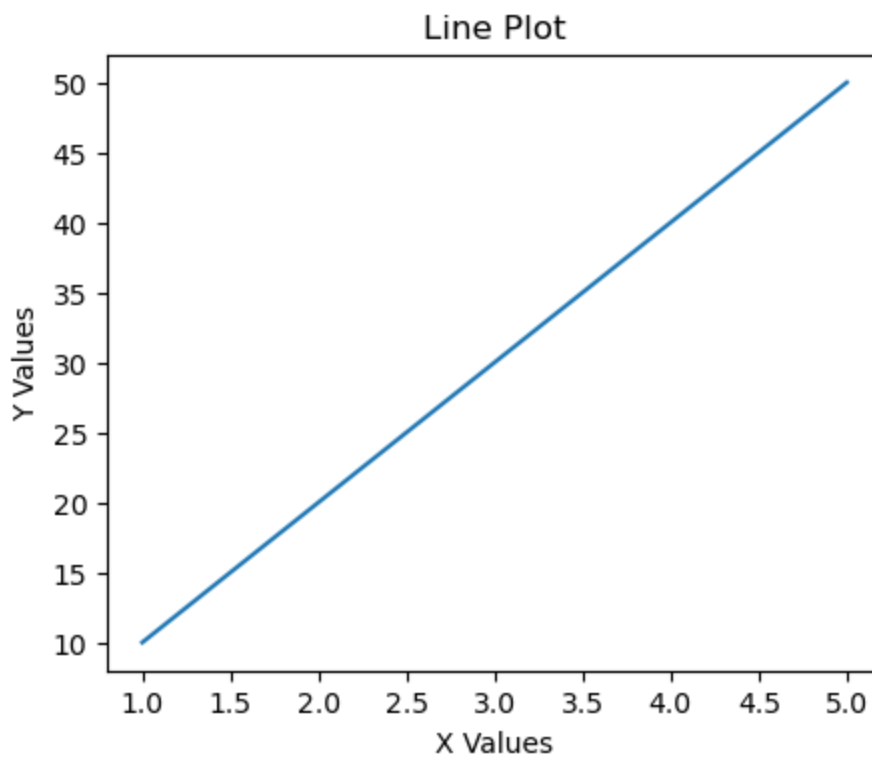
data = [10,20,20,30,30,30,40,50,50]

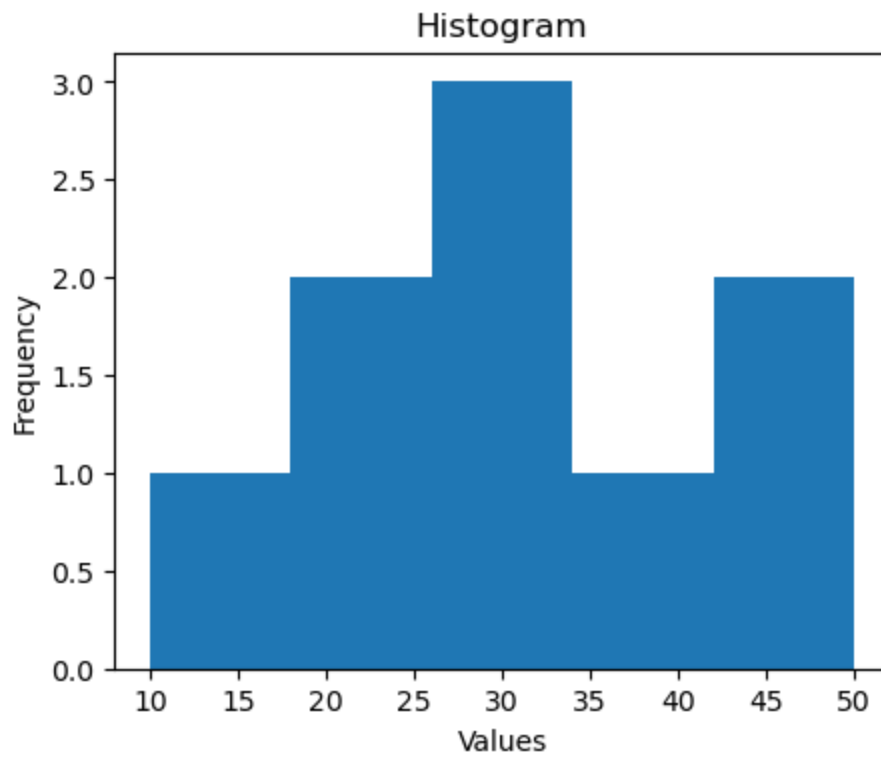
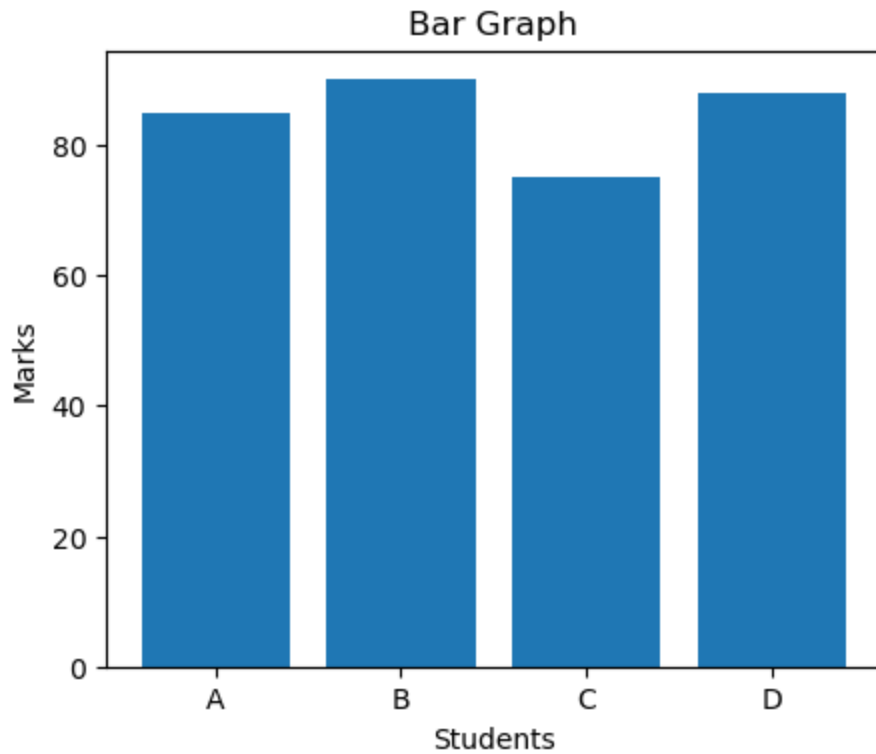
plt.figure(figsize=(5,4))

plt.hist(data, bins=5)

plt.title("Histogram")
plt.xlabel("Values")
plt.ylabel("Frequency")

plt.show()
```





Practical 16

```
In [22]: import matplotlib.pyplot as plt

# -----SCATTER PLOT-----

x = [1, 2, 3, 4, 5]
```

```
y = [10, 25, 30, 45, 50]

plt.figure(figsize=(8,4))

plt.scatter(x,y)

plt.title("Scatter Plot")
plt.xlabel("X Values")
plt.ylabel("Y Values")

plt.show()

# -----SUBPLOTS-----

x1 = [1, 2, 3, 4]
y1 = [10, 20, 30, 40]

x2 = [1, 2, 3, 4]
y2 = [40, 30, 20, 10]

plt.figure(figsize=(8,4))

# First Subplot
plt.subplot(1,2,1)

plt.plot(x1,y1)

plt.title("Line Plot")

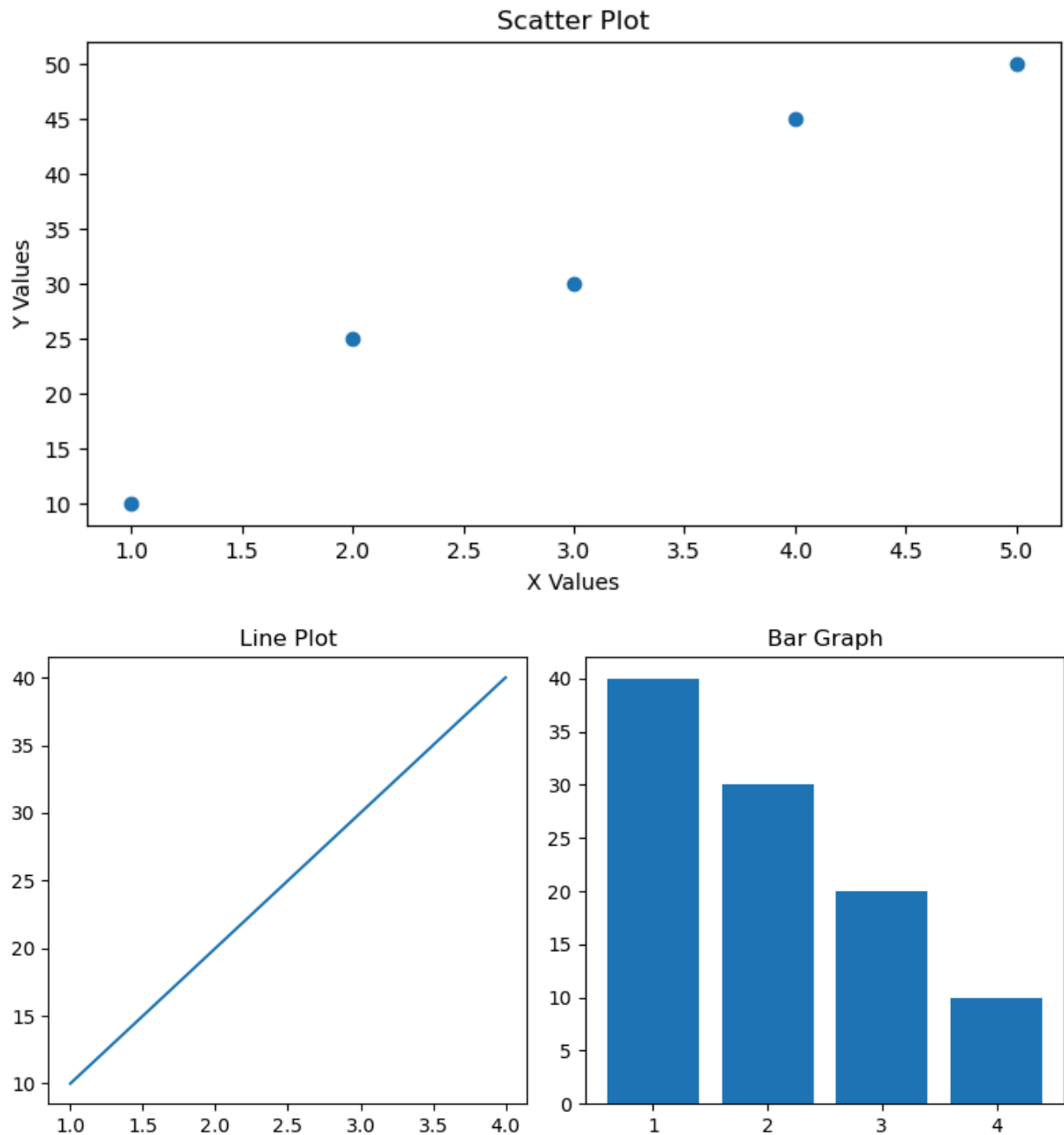
# Second Subplot
plt.subplot(1,2,2)

plt.bar(x2,y2)

plt.title("Bar Graph")

plt.tight_layout()

plt.show()
```



Practical 17

```
In [5]: # PRACTICAL 17
# Load and Analyze a CSV Dataset using Pandas

import pandas as pd
import csv

# Loading CSV File
file_path = r"students.csv"

# 1. Load the CSV dataset
#df = pd.read_csv(file_path)
```

```
df = pd.read_csv(file_path, sep=";", quoting=csv.QUOTE_NONE)

# 2. Display complete dataset
print("===== COMPLETE DATASET =====")
print(df)

# 3. Display first 5 rows
print("\n===== FIRST 5 ROWS =====")
print(df.head())

# 4. Display last 5 rows
print("\n===== LAST 5 ROWS =====")
print(df.tail())

# 5. Display number of rows and columns
print("\n===== DATASET SHAPE =====")
print(df.shape)

# 6. Display column names
print("\n===== COLUMN NAMES =====")
print(df.columns)

# 7. Display data types
print("\n===== DATA TYPES =====")
print(df.dtypes)

# 8. Display dataset information
print("\n===== DATASET INFORMATION =====")
df.info()

# 9. Display statistical summary
print("\n===== STATISTICAL SUMMARY =====")
print(df.describe())

# 10. Check missing values
print("\n===== MISSING VALUES =====")
print(df.isnull().sum())

# 11. Display a specific column
print("\n===== MARKS COLUMN =====")
print(df["Marks"])

# 12. Filter students with marks greater than 80
print("\n===== STUDENTS WITH MARKS > 80 =====")
print(df[df["Marks"] > 80])

# 13. Sort dataset by Marks
print("\n===== DATA SORTED BY MARKS =====")
print(df.sort_values(by="Marks"))

# 14. Calculate average marks by department
print("\n===== AVERAGE MARKS BY DEPARTMENT =====")
print(df.groupby("Department")["Marks"].mean())
```

===== COMPLETE DATASET =====

	"ID	Name	Department	Marks	City"
0	"101	Sakshi	Data Science	95	Chennai"
1	"102	Rahul	AI	88	Delhi"
2	"103	Ankit	Data Science	91	Mumbai"
3	"104	Priya	Cyber Security	84	Bangalore"
4	"105	Riya	AI	90	Hyderabad"
5	"106	Amit	Data Science	76	Pune"
6	"107	Neha	Cyber Security	82	Kolkata"
7	"108	Karan	AI	89	Chennai"
8	"109	Simran	Data Science	93	Delhi"
9	"110	Arjun	Cyber Security	85	Mumbai"

===== FIRST 5 ROWS =====

	"ID	Name	Department	Marks	City"
0	"101	Sakshi	Data Science	95	Chennai"
1	"102	Rahul	AI	88	Delhi"
2	"103	Ankit	Data Science	91	Mumbai"
3	"104	Priya	Cyber Security	84	Bangalore"
4	"105	Riya	AI	90	Hyderabad"

===== LAST 5 ROWS =====

	"ID	Name	Department	Marks	City"
5	"106	Amit	Data Science	76	Pune"
6	"107	Neha	Cyber Security	82	Kolkata"
7	"108	Karan	AI	89	Chennai"
8	"109	Simran	Data Science	93	Delhi"
9	"110	Arjun	Cyber Security	85	Mumbai"

===== DATASET SHAPE =====

(10, 5)

===== COLUMN NAMES =====

Index(['ID', 'Name', 'Department', 'Marks', 'City"', dtype='str')

===== DATA TYPES =====

```
"ID          str
Name         str
Department   str
Marks        int64
City"        str
dtype: object
```

===== DATASET INFORMATION =====

```
<class 'pandas.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 5 columns):
#   Column          Non-Null Count  Dtype
---  -
0   "ID              10 non-null    str
1   Name             10 non-null    str
2   Department       10 non-null    str
3   Marks            10 non-null    int64
4   City"            10 non-null    str
dtypes: int64(1), str(4)
memory usage: 532.0 bytes
```

===== STATISTICAL SUMMARY =====

```

Marks
count    10.000000
mean     87.300000
std       5.657836
min       76.000000
25%      84.250000
50%      88.500000
75%      90.750000
max       95.000000

```

===== MISSING VALUES =====

```

"ID        0
Name       0
Department 0
Marks      0
City"      0
dtype: int64

```

===== MARKS COLUMN =====

```

0    95
1    88
2    91
3    84
4    90
5    76
6    82
7    89
8    93
9    85
Name: Marks, dtype: int64

```

===== STUDENTS WITH MARKS > 80 =====

	"ID	Name	Department	Marks	City"
0	"101	Sakshi	Data Science	95	Chennai"
1	"102	Rahul	AI	88	Delhi"
2	"103	Ankit	Data Science	91	Mumbai"
3	"104	Priya	Cyber Security	84	Bangalore"
4	"105	Riya	AI	90	Hyderabad"
6	"107	Neha	Cyber Security	82	Kolkata"
7	"108	Karan	AI	89	Chennai"
8	"109	Simran	Data Science	93	Delhi"
9	"110	Arjun	Cyber Security	85	Mumbai"

===== DATA SORTED BY MARKS =====

	"ID	Name	Department	Marks	City"
5	"106	Amit	Data Science	76	Pune"
6	"107	Neha	Cyber Security	82	Kolkata"
3	"104	Priya	Cyber Security	84	Bangalore"
9	"110	Arjun	Cyber Security	85	Mumbai"
1	"102	Rahul	AI	88	Delhi"
7	"108	Karan	AI	89	Chennai"
4	"105	Riya	AI	90	Hyderabad"
2	"103	Ankit	Data Science	91	Mumbai"
8	"109	Simran	Data Science	93	Delhi"

```
0  "101  Sakshi      Data Science      95      Chennai"

===== AVERAGE MARKS BY DEPARTMENT =====
Department
AI                89.000000
Cyber Security    83.666667
Data Science      88.750000
Name: Marks, dtype: float64
```

Practical 18

```
In [7]: # PRACTICAL 18
        # Basic Data Cleaning and Preprocessing

import pandas as pd
import numpy as np

# Loading CSV File
file_path = r"students.csv"

# 1. Load the CSV dataset
#df = pd.read_csv(file_path)
df = pd.read_csv(file_path, sep=";", quoting=csv.QUOTE_NONE)

# 2. Display original dataset
print("===== ORIGINAL DATASET =====")
print(df)

# 3. Display dataset information
print("\n===== ORIGINAL INFORMATION =====")
df.info()

# 4. Check missing values
print("\n===== MISSING VALUES =====")
print(df.isnull().sum())

# 5. Fill missing Age values with mean
if "Age" in df.columns:
    df["Age"] = df["Age"].fillna(df["Age"].mean())

# 6. Fill missing Marks values with mean
if "Marks" in df.columns:
    df["Marks"] = df["Marks"].fillna(df["Marks"].mean())

# 7. Fill missing City values with "Unknown"
if "City" in df.columns:
    df["City"] = df["City"].fillna("Unknown")

# 8. Remove duplicate records
df = df.drop_duplicates()

print("\n===== AFTER REMOVING DUPLICATES =====")
print(df)
```

```
# 9. Remove extra spaces from Name column
if "Name" in df.columns:
    df["Name"] = df["Name"].str.strip()

# 10. Convert Name column to lowercase
if "Name" in df.columns:
    df["Name"] = df["Name"].str.lower()

# 11. Convert City column to lowercase
if "City" in df.columns:
    df["City"] = df["City"].str.lower()

# 12. Convert Age into integer
if "Age" in df.columns:
    df["Age"] = df["Age"].astype(int)

# 13. Rename Marks column as Score
if "Marks" in df.columns:
    df.rename(columns={"Marks": "Score"}, inplace=True)

# 14. Sort dataset by Score
if "Score" in df.columns:
    df = df.sort_values(by="Score", ascending=False)

# 15. Reset the index
df = df.reset_index(drop=True)

# 16. Display cleaned dataset
print("\n===== CLEANED DATASET =====")
print(df)

# 17. Check missing values after cleaning
print("\n===== MISSING VALUES AFTER CLEANING =====")
print(df.isnull().sum())

# 18. Display final dataset information
print("\n===== FINAL DATASET INFORMATION =====")
df.info()
```

===== ORIGINAL DATASET =====

	"ID	Name	Department	Marks	City"
0	"101	Sakshi	Data Science	95	Chennai"
1	"102	Rahul	AI	88	Delhi"
2	"103	Ankit	Data Science	91	Mumbai"
3	"104	Priya	Cyber Security	84	Bangalore"
4	"105	Riya	AI	90	Hyderabad"
5	"106	Amit	Data Science	76	Pune"
6	"107	Neha	Cyber Security	82	Kolkata"
7	"108	Karan	AI	89	Chennai"
8	"109	Simran	Data Science	93	Delhi"
9	"110	Arjun	Cyber Security	85	Mumbai"

===== ORIGINAL INFORMATION =====

```
<class 'pandas.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 5 columns):
#   Column          Non-Null Count  Dtype
---  -
0    "ID              10 non-null    str
1    Name             10 non-null    str
2    Department        10 non-null    str
3    Marks            10 non-null    int64
4    City"           10 non-null    str
dtypes: int64(1), str(4)
memory usage: 532.0 bytes
```

===== MISSING VALUES =====

```
"ID          0
Name         0
Department   0
Marks        0
City"        0
dtype: int64
```

===== AFTER REMOVING DUPLICATES =====

	"ID	Name	Department	Marks	City"
0	"101	Sakshi	Data Science	95	Chennai"
1	"102	Rahul	AI	88	Delhi"
2	"103	Ankit	Data Science	91	Mumbai"
3	"104	Priya	Cyber Security	84	Bangalore"
4	"105	Riya	AI	90	Hyderabad"
5	"106	Amit	Data Science	76	Pune"
6	"107	Neha	Cyber Security	82	Kolkata"
7	"108	Karan	AI	89	Chennai"
8	"109	Simran	Data Science	93	Delhi"
9	"110	Arjun	Cyber Security	85	Mumbai"

===== CLEANED DATASET =====

	"ID	Name	Department	Score	City"
0	"101	sakshi	Data Science	95	Chennai"
1	"109	simran	Data Science	93	Delhi"
2	"103	ankit	Data Science	91	Mumbai"
3	"105	riya	AI	90	Hyderabad"
4	"108	karan	AI	89	Chennai"
5	"102	rahul	AI	88	Delhi"

```

6  "110    arjun  Cyber Security    85    Mumbai"
7  "104    priya  Cyber Security    84    Bangalore"
8  "107     neha  Cyber Security    82    Kolkata"
9  "106     amit   Data Science    76     Pune"

```

===== MISSING VALUES AFTER CLEANING =====

```

"ID          0
Name         0
Department   0
Score        0
City"        0
dtype: int64

```

===== FINAL DATASET INFORMATION =====

```

<class 'pandas.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 5 columns):
 #   Column          Non-Null Count  Dtype
---  -
 0   "ID             10 non-null    str
 1   Name            10 non-null    str
 2   Department       10 non-null    str
 3   Score           10 non-null    int64
 4   City"           10 non-null    str
dtypes: int64(1), str(4)
memory usage: 532.0 bytes

```

Practical 19

```

In [9]: # PRACTICAL 19
        # Calculate Basic Statistics from a Dataset

import pandas as pd
import numpy as np

# Loading CSV File
file_path = r"students.csv"

# 1. Load the CSV dataset
#df = pd.read_csv(file_path)
df = pd.read_csv(file_path, sep=";", quoting=csv.QUOTE_NONE)

# 2. Display the dataset
print("===== DATASET =====")
print(df)

# 3. Select Marks column
marks = df["Marks"]

# 4. Remove missing values
marks = marks.dropna()

# 5. Count number of observations

```

```
count = marks.count()

# 6. Calculate total marks
total = marks.sum()

# 7. Calculate Mean
mean = marks.mean()

# 8. Calculate Median
median = marks.median()

# 9. Calculate Mode
mode = marks.mode()

# 10. Calculate Standard Deviation
std = marks.std()

# 11. Calculate Variance
variance = marks.var()

# 12. Find Minimum
minimum = marks.min()

# 13. Find Maximum
maximum = marks.max()

# 14. Calculate Range
data_range = maximum - minimum

# 15. Calculate First Quartile
q1 = marks.quantile(0.25)

# 16. Calculate Third Quartile
q3 = marks.quantile(0.75)

# 17. Display results
print("\n===== BASIC STATISTICS =====")

print("Number of Observations :", count)
print("Total Marks            :", total)
print("Mean                    :", mean)
print("Median                   :", median)
print("Mode                      :", mode.tolist())
print("Standard Deviation        :", std)
print("Variance                   :", variance)
print("Minimum                   :", minimum)
print("Maximum                   :", maximum)
print("Range                     :", data_range)
print("First Quartile (Q1)       :", q1)
print("Third Quartile (Q3)       :", q3)
```

===== DATASET =====

	"ID	Name	Department	Marks	City"
0	"101	Sakshi	Data Science	95	Chennai"
1	"102	Rahul	AI	88	Delhi"
2	"103	Ankit	Data Science	91	Mumbai"
3	"104	Priya	Cyber Security	84	Bangalore"
4	"105	Riya	AI	90	Hyderabad"
5	"106	Amit	Data Science	76	Pune"
6	"107	Neha	Cyber Security	82	Kolkata"
7	"108	Karan	AI	89	Chennai"
8	"109	Simran	Data Science	93	Delhi"
9	"110	Arjun	Cyber Security	85	Mumbai"

===== BASIC STATISTICS =====

Number of Observations : 10
 Total Marks : 873
 Mean : 87.3
 Median : 88.5
 Mode : [76, 82, 84, 85, 88, 89, 90, 91, 93, 95]
 Standard Deviation : 5.657836257007718
 Variance : 32.011111111111106
 Minimum : 76
 Maximum : 95
 Range : 19
 First Quartile (Q1) : 84.25
 Third Quartile (Q3) : 90.75

Practical 20

```
In [4]: # Data Visualization using Seaborn
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Creating Sample Dataset
data = {
    "Maths": [85, 90, 78, 92, 88],
    "Science": [80, 85, 75, 95, 89],
    "English": [78, 83, 72, 90, 84]
}

# Creating DataFrame
df = pd.DataFrame(data)

print("Dataset:")
print(df)

# -----HEATMAP-----

plt.figure(figsize=(5,4))

sns.heatmap(df, annot=True)

plt.title("Heatmap")
```

```
plt.show()

#-----BOXPLOT-----

plt.figure(figsize=(5,4))

sns.boxplot(data=df)

plt.title("Boxplot")

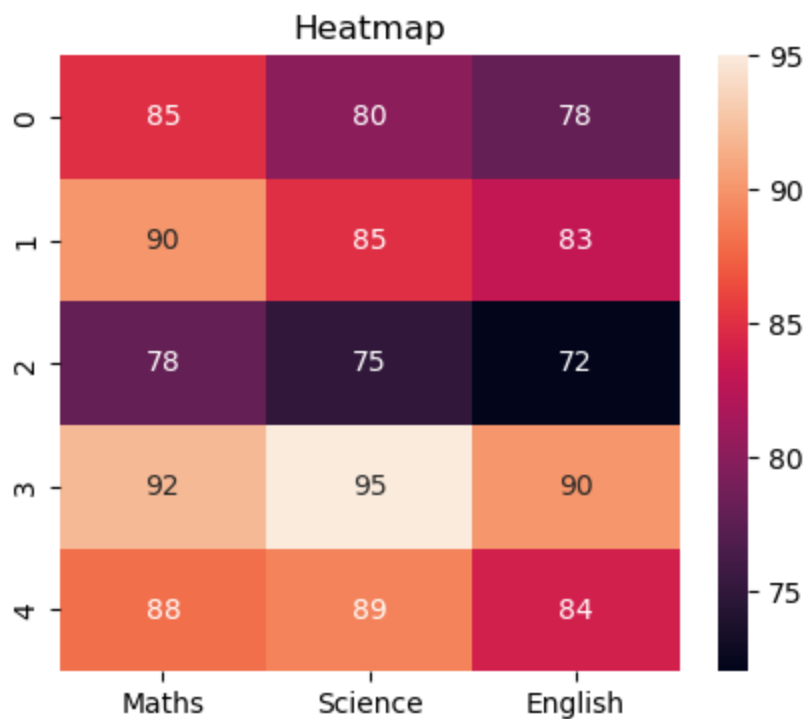
plt.show()

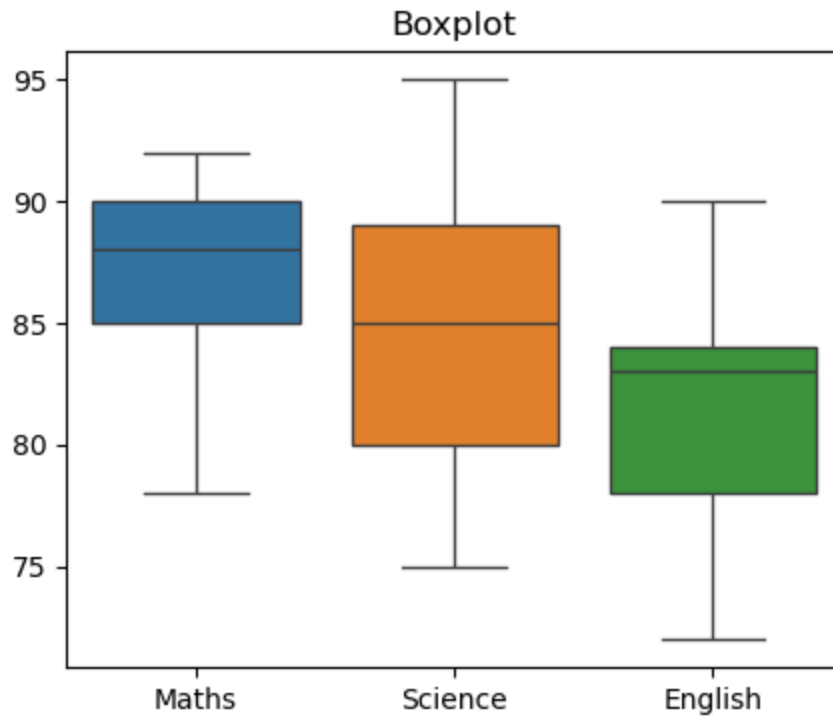
#-----PAIR PLOT-----

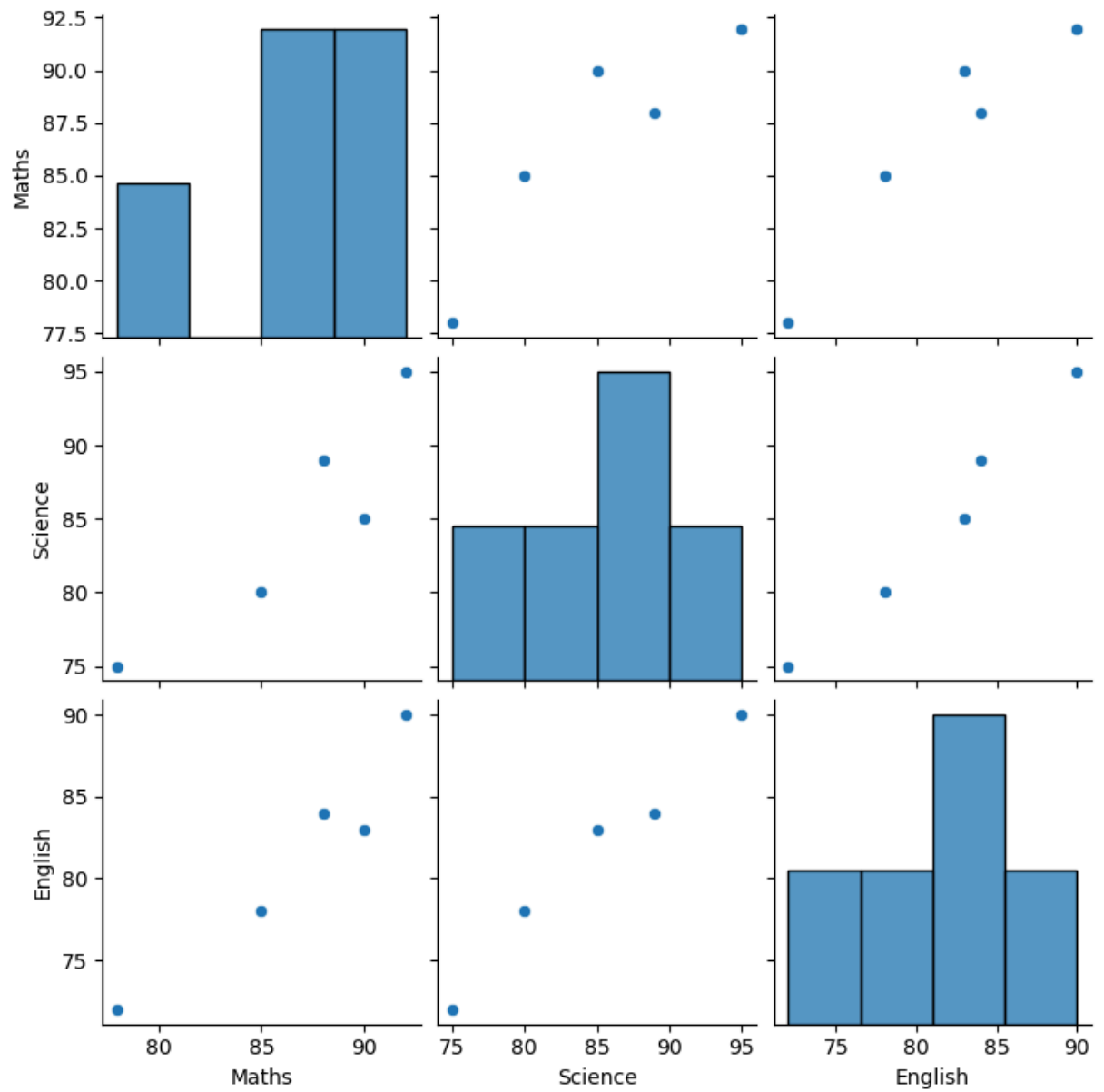
sns.pairplot(df)
plt.show()
```

Dataset:

	Maths	Science	English
0	85	80	78
1	90	85	83
2	78	75	72
3	92	95	90
4	88	89	84







In []: